ARM Linker File

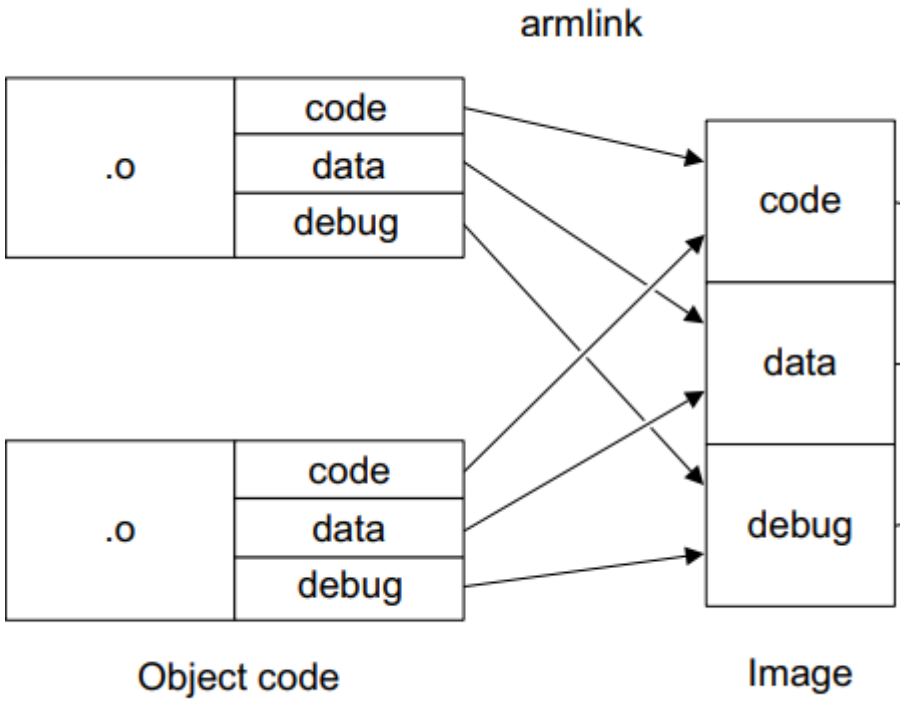
10 Aug 2022

OUTLINE

1. Overview Linker
2. Create the user section
3. Scatter file
4. Memory map

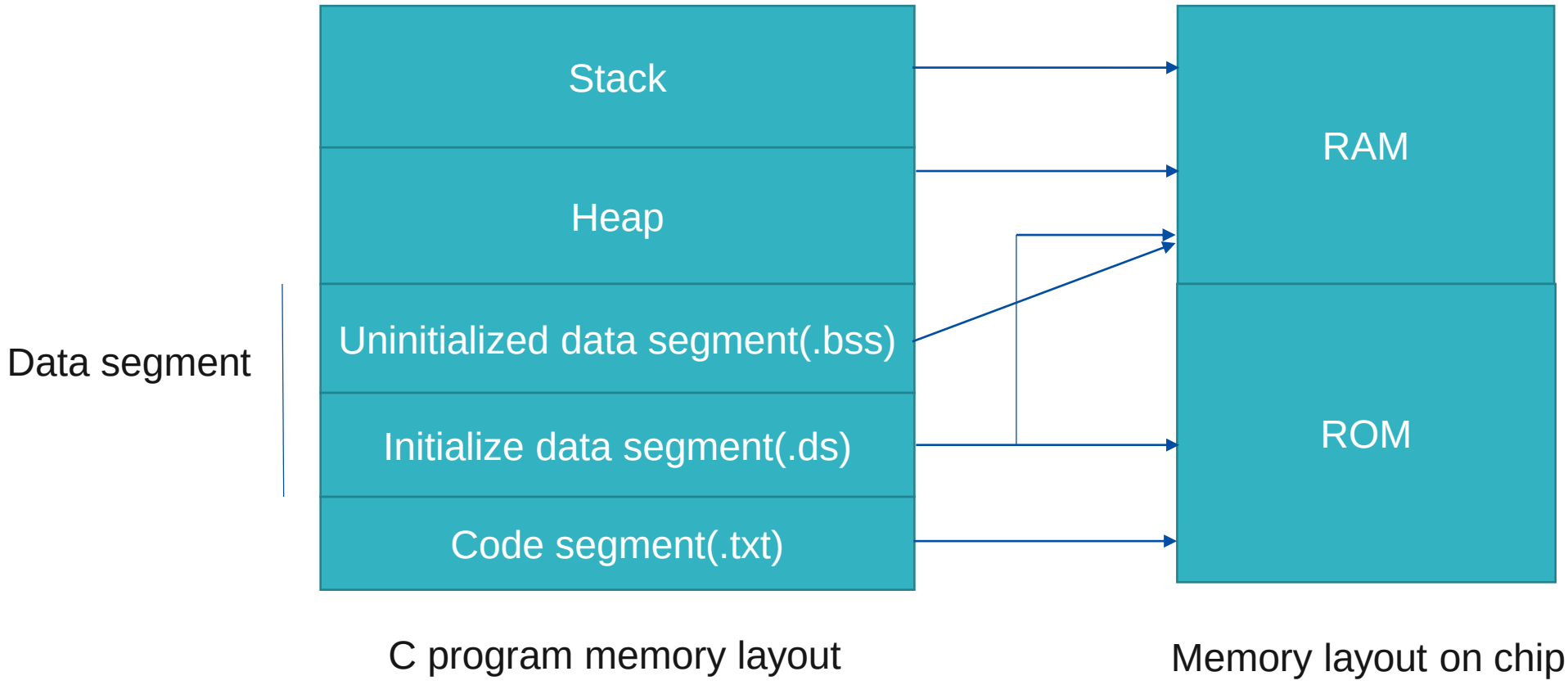
1. Overview Linker

- What is the purpose of linker?
- The linker takes object files that a compiler or assembler produces and combines them into an executable image.



1. Overview Linker

- The relation of C program memory layout and the Memory layout on chip



1. Overview Linker

- For example, after disassembly of the object file of main.o and lib.o, we can see that the f_App_Sum function and main function start at address 0, it can not run on the target (MCU), so the linker assigns the data and code to the specific address, which can run on the target(MCU).

```
$t.0
[Anonymous symbol #3]
f_App_Sum
    0x00000000:    b083        ..      SUB     sp,sp,#0xc
    0x00000002:    460b        .F      MOV     r3,r1
    0x00000004:    4684        .F      MOV     r12,r0
    0x00000006:    f88d000b    ....    STRB    r0,[sp,#0xb]
    0x0000000a:    f88d100a    ....    STRB    r1,[sp,#0xa]
    0x0000000e:    9201        ..      STR     r2,[sp,#4]
    0x00000010:    f89d000b    ....    LDRB    r0,[sp,#0xb]
    0x00000014:    f89d100a    ....    LDRB    r1,[sp,#0xa]
    0x00000018:    4408        .D      ADD     r0,r0,r1
    0x0000001a:    f88d0003    ....    STRB    r0,[sp,#3]
    0x0000001e:    f89d0003    ....    LDRB    r0,[sp,#3]
    0x00000022:    9901        ..      LDR     r1,[sp,#4]
    0x00000024:    7008        .p      STRB    r0,[r1,#0]
    0x00000026:    b003        ..      ADD     sp,sp,#0xc
    0x00000028:    4770        pG      BX      lr
```

```
$t.0
[Anonymous symbol #4]
main
    0x00000000:    b580        ..      PUSH    {r7,lr}
    0x00000002:    b082        ..      SUB     sp,sp,#8
    0x00000004:    2000        .       MOVS    r0,#0
    0x00000006:    9001        ..      STR     r0,[sp,#4]
    0x00000008:    f2400000    @...    MOVW    r0,#:LOWER16:xnl_Num
    0x0000000c:    f2c00000    ....    MOVT    r0,#:UPPER16:xnl_Num
    0x00000010:    7800        .x      LDRB    r0,[r0,#0]
    0x00000012:    f88d0003    ....    STRB    r0,[sp,#3]
    0x00000016:    f04f0109    O...    MOV     r1,#9
    0x0000001a:    4608        .F      MOV     r0,r1
    0x0000001c:    2003        .       MOVS    r0,#3
    0x0000001e:    2105        .!      MOVS    r1,#5
    0x00000020:    f10d0203    ....    ADD     r2,sp,#3
    0x00000024:    f7fffffe    ....    BL      f_App_Sum
    0x00000028:    f89d0003    ....    LDRB    r0,[sp,#3]
    0x0000002c:    2821        !(      CMP     r0,#0x21
    0x0000002e:    d102        ..      BNE     {pc}+0x8 ; 0x36
    0x00000030:    e7ff        ..      B       {pc}+0x2 ; 0x32
    0x00000032:    e7ff        ..      B       {pc}+0x2 ; 0x34
    0x00000034:    e7fe        ..      B       {pc} ; 0x34
    0x00000036:    2000        .       MOVS    r0,#0
    0x00000038:    b002        ..      ADD     sp,sp,#8
    0x0000003a:    bd80        ..      POP     {r7,pc}
```

Linker

```
$t.0
f_App_Sum
    0x00008328:    b083        ..      SUB     sp,sp,#0xc
    0x0000832a:    460b        .F      MOV     r3,r1
    0x0000832c:    4684        .F      MOV     r12,r0
    0x0000832e:    f88d000b    ....    STRB    r0,[sp,#0xb]
    0x00008332:    f88d100a    ....    STRB    r1,[sp,#0xa]
    0x00008336:    9201        ..      STR     r2,[sp,#4]
    0x00008338:    f89d000b    ....    LDRB    r0,[sp,#0xb]
    0x0000833c:    f89d100a    ....    LDRB    r1,[sp,#0xa]
    0x00008340:    4408        .D      ADD     r0,r0,r1
    0x00008342:    f88d0003    ....    STRB    r0,[sp,#3]
    0x00008346:    f89d0003    ....    LDRB    r0,[sp,#3]
    0x0000834a:    9901        ..      LDR     r1,[sp,#4]
    0x0000834c:    7008        .p      STRB    r0,[r1,#0]
    0x0000834e:    b003        ..      ADD     sp,sp,#0xc
    0x00008350:    4770        pG      BX      lr
    0x00008352:    0000        ..      MOVS    r0,r0
main
    0x00008354:    b580        ..      PUSH    {r7,lr}
    0x00008356:    b082        ..      SUB     sp,sp,#8
    0x00008358:    2000        .       MOVS    r0,#0
    0x0000835a:    9001        ..      STR     r0,[sp,#4]
    0x0000835c:    f24830bc    H..0    MOV     r0,#0x83bc
    0x00008360:    f2c00000    ....    MOVT    r0,#0
    0x00008364:    7800        .x      LDRB    r0,[r0,#0]
```

1. Overview Linker – Scatter file

- The linker also uses a memory description to assign the input code and data from the object files to the required addresses in the image.
- The linker has a built-in memory description that it uses by default. However, you can override this default memory description with command-line options or with a scatter file. The method that you use depends how much you want to control the placement of the various output sections in the image:
  - Allow the linker to automatically place the output sections using the default memory map for the specified linking model. armlink uses default locations for the RO, RW, *execute-only* (XO), and ZI output sections.
  - Use the memory map related command-line options to specify the locations of the RO, RW, XO, and ZI output sections.
  - Use a scatter file if you want to have the most control over where the linker places various parts of your image. For example, you can place individual functions at specific addresses or certain data structures at peripheral addresses

1. Overview Linker – Scatter file

❖ **What you can control with a scatter file**

A scatter file gives you the ability to control where the linker places different parts of your image for your particular target.

You can control:

• The location and size of various memory regions that are mapped to ROM, RAM, and FLASH.

• The location of individual functions and variables, and code from the Arm standard C and C++ libraries.

• The placement of sections that contain individual functions or variables, or code from the Arm standard C and C++ libraries.

• The priority ordering of memory areas for placing unassigned sections, to ensure that they get filled in a particular order.

• The location and size of empty regions of memory, such as memory to use for stack and heap.

1. Overview Linker – The Image after linking

- The *.map file displays the image memory map, and contains the address and the size of each load region, execution region, and input section in the image, including linker-generated input sections. It also shows how RW data compression is applied.
- The memory map of the Image is shown below:

Infor.map

Compiler > Infor.map

```
939
940
941  ==============================================================================
942
943  Memory Map of the image
944
945    Image Entry point : 0x00008355
946
947    Load Region LR_1 (Base: 0x00008000, Size: 0x000003c4, Max: 0xffffffff, ABSOLUTE)
948
949      Execution Region ER_RO (Exec base: 0x00008000, Load base: 0x00008000, Size: 0x000003bc, Max: 0xffffffff, ABSOLUTE)
950
951        Exec Addr    Load Addr    Size         Type   Attr      Idx    E Section Name        Object
952
953        0x00008000   0x00008000   0x00000008   Code   RO         10  * !!!main             c_wu.l(__main.o)
954        0x00008008   0x00008008   0x00000034   Code   RO        177    !!!scatter          c_wu.l(__scatter.o)
955        0x0000803c   0x0000803c   0x0000001c   Code   RO        179    !!handler_zi        c_wu.l(__scatter_zi.o)
956        0x00008058   0x00008058   0x00000002   Code   RO         37    .ARM.Collect$$libinit$$00000000  c_wu.l(libinit.o)
957        0x0000805a   0x0000805a   0x00000004   Code   RO         47    .ARM.Collect$$libinit$$00000001  c_wu.l(libinit2.o)
958        0x0000805e   0x0000805e   0x00000000   Code   RO         50    .ARM.Collect$$libinit$$00000004  c_wu.l(libinit2.o)
959        0x0000805e   0x0000805e   0x00000000   Code   RO         53    .ARM.Collect$$libinit$$0000000A  c_wu.l(libinit2.o)
960        0x0000805e   0x0000805e   0x00000000   Code   RO         55    .ARM.Collect$$libinit$$0000000C  c_wu.l(libinit2.o)
961        0x0000805e   0x0000805e   0x00000000   Code   RO         57    .ARM.Collect$$libinit$$0000000E  c_wu.l(libinit2.o)
962        0x0000805e   0x0000805e   0x00000000   Code   RO         60    .ARM.Collect$$libinit$$00000011  c_wu.l(libinit2.o)
963        0x0000805e   0x0000805e   0x00000000   Code   RO         62    .ARM.Collect$$libinit$$00000013  c_wu.l(libinit2.o)
964        0x0000805e   0x0000805e   0x00000000   Code   RO         64    .ARM.Collect$$libinit$$00000015  c_wu.l(libinit2.o)
965        0x0000805e   0x0000805e   0x00000000   Code   RO         66    .ARM.Collect$$libinit$$00000017  c_wu.l(libinit2.o)
966        0x0000805e   0x0000805e   0x00000000   Code   RO         68    .ARM.Collect$$libinit$$00000019  c_wu.l(libinit2.o)
967        0x0000805e   0x0000805e   0x00000000   Code   RO         70    .ARM.Collect$$libinit$$0000001B  c_wu.l(libinit2.o)
968        0x0000805e   0x0000805e   0x00000000   Code   RO         72    .ARM.Collect$$libinit$$0000001D  c_wu.l(libinit2.o)
969        0x0000805e   0x0000805e   0x00000000   Code   RO         74    .ARM.Collect$$libinit$$0000001F  c_wu.l(libinit2.o)
970        0x0000805e   0x0000805e   0x00000000   Code   RO         76    .ARM.Collect$$libinit$$00000021  c_wu.l(libinit2.o)
971        0x0000805e   0x0000805e   0x00000000   Code   RO         78    .ARM.Collect$$libinit$$00000023  c_wu.l(libinit2.o)
972        0x0000805e   0x0000805e   0x00000000   Code   RO         80    .ARM.Collect$$libinit$$00000025  c_wu.l(libinit2.o)
973        0x0000805e   0x0000805e   0x00000008   Code   RO         82    .ARM.Collect$$libinit$$00000027  c_wu.l(libinit2.o)
974        0x00008066   0x00008066   0x00000000   Code   RO         86    .ARM.Collect$$libinit$$0000002E  c_wu.l(libinit2.o)
975        0x00008066   0x00008066   0x00000000   Code   RO         88    .ARM.Collect$$libinit$$00000030  c_wu.l(libinit2.o)
976        0x00008066   0x00008066   0x00000000   Code   RO         90    .ARM.Collect$$libinit$$00000032  c_wu.l(libinit2.o)
```

2. Create the user section

- Method 1: using Compiler-specific __attribute__
- The syntax: __attribute__((section("*name*")))
- For example:
  - Create the variable with default section:

```
12    static volatile t_NumType xnl_Num = { 1U, 2U, 3U };
13

** Section #7 '.data.xnl_Num' (SHT_PROGBITS) [SHF_ALLOC + SHF_WRITE]
    Size   : 8 bytes (alignment 4)
    Address: 0x00000000

    0x000000:  01 00 02 00 03 00 00 00                        ........
```

  - Create the variable with user section:

```
12    __attribute__((section(".userData"))) static volatile t_NumType xnl_Num = { 1U, 2U, 3U };
13

** Section #7 '.userData' (SHT_PROGBITS) [SHF_ALLOC + SHF_WRITE]
    Size   : 8 bytes (alignment 4)
    Address: 0x00000000

    0x000000:  01 00 02 00 03 00 00 00                        ........
```

2. Create the user section

- Method 2: using Compiler-specific pragmas
  - The syntax: #pragma clang section [*section_type_list*]
    Where:
    - *section_type_list:* specifies an optional list of section names to be used for subsequent functions, global variables, or static variables.
    - The syntax of *section_type_list* is:
      section_type="*name*"[section_type="*name*"]
    - The user can revert to the default section name by specifying an empty string, "", for *name*.
    - Valid section types are:
      • bss.
      • data.
      • rodata.
      • text.

2. Create the user section

- Method 2: using Compiler-specific pragmas
  - Example:

```
// __attribute__((section(".userData"))) static volatile t_NumType xnl_Num = { 1U, 2U, 3U };

#pragma clang section data = ".structData"
static volatile t_NumType xnl_Num = { 1U, 2U, 3U };
#pragma clang data = ""
```

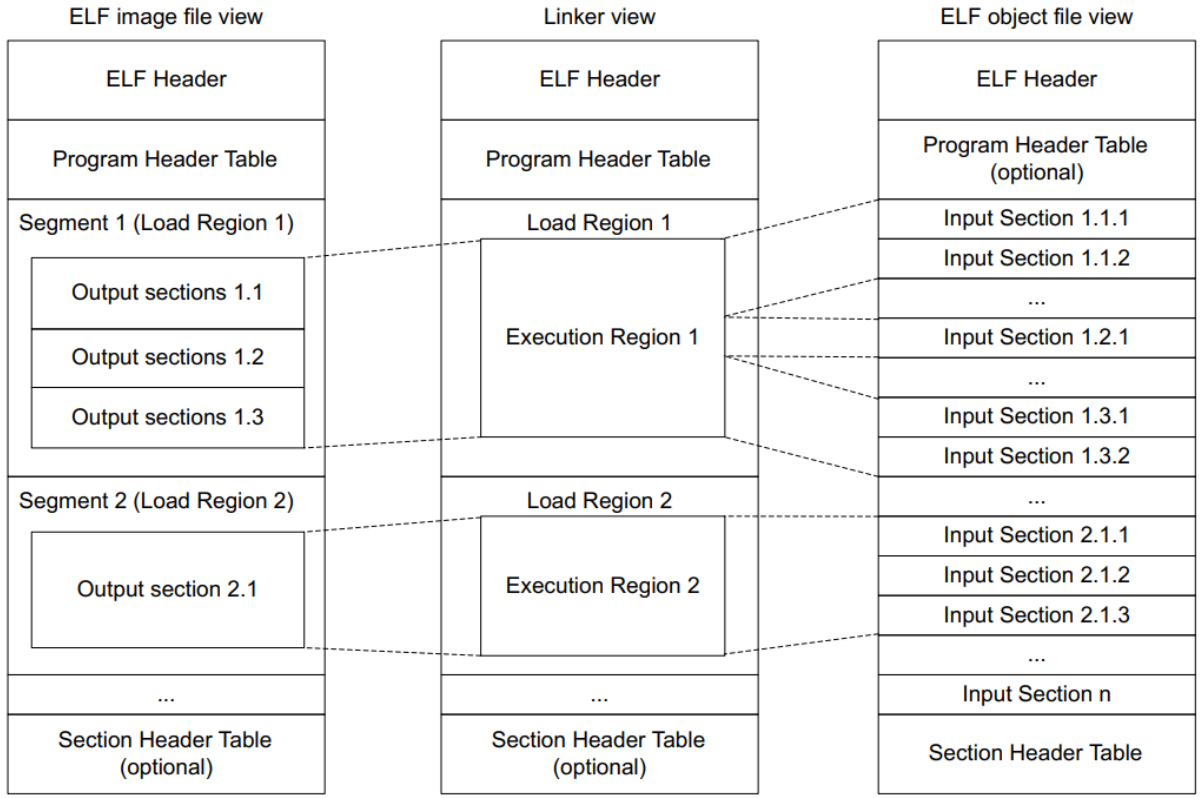
```
** Section #7 '.structData' (SHT_PROGBITS) [SHF_ALLOC + SHF_WRITE]
    Size   : 8 bytes (alignment 4)
    Address: 0x00000000

    0x000000:   01 00 02 00 03 00 00 00                        ........
```

3. Scatter file – The struct of the Image

- An Arm ELF image contains sections, regions, and segments, and each link stage has a different view of the image.
- The structure of an image is defined by the:
  - Number of its constituent regions and output sections.
  - Positions in memory of these regions and sections when the image is loaded.
  - Positions in memory of these regions and sections when the image executes.

- The following figure shows the relationship between the views at each link stage:

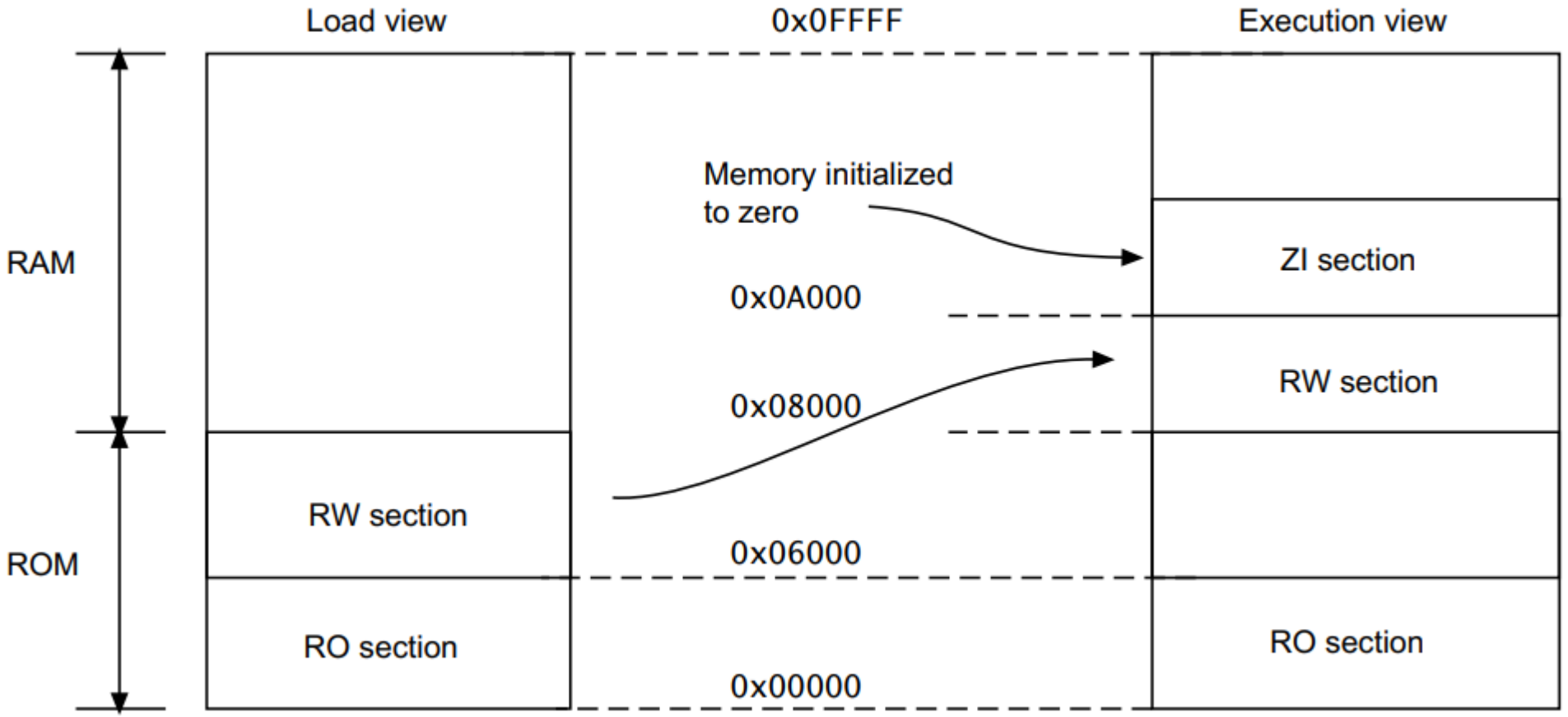


3. Scatter file – Image Conception

- **Input section:** An input section is an individual section from an input object file. It contains code, initialized data, or describes a fragment of memory that is not initialized or that must be set to zero before the image can execute. These properties are represented by attributes such as RO, RW, XO, and ZI.
- **Output section:** An output section is a group of input sections that have the same RO, RW, XO, or ZI attribute, and that are placed contiguously in memory by the linker. An output section has the same attributes as its constituent input sections. Within an output section, the input sections are sorted according to the section placement rules.
- **Region:** A region contains up to three output sections depending on the contents and the number of sections with different attributes.
- **Program segment:** A program segment corresponds to a load region and contains execution regions. Program
  segments hold information such as text and data.

3. Scatter file – Load view and execution view

- **Load view** Describes each image region and section in terms of the address where it is located when the image is loaded into memory, that is, the location before image execution starts.
- **Execution view** Describes each image region and section in terms of the address where it is located during image execution.



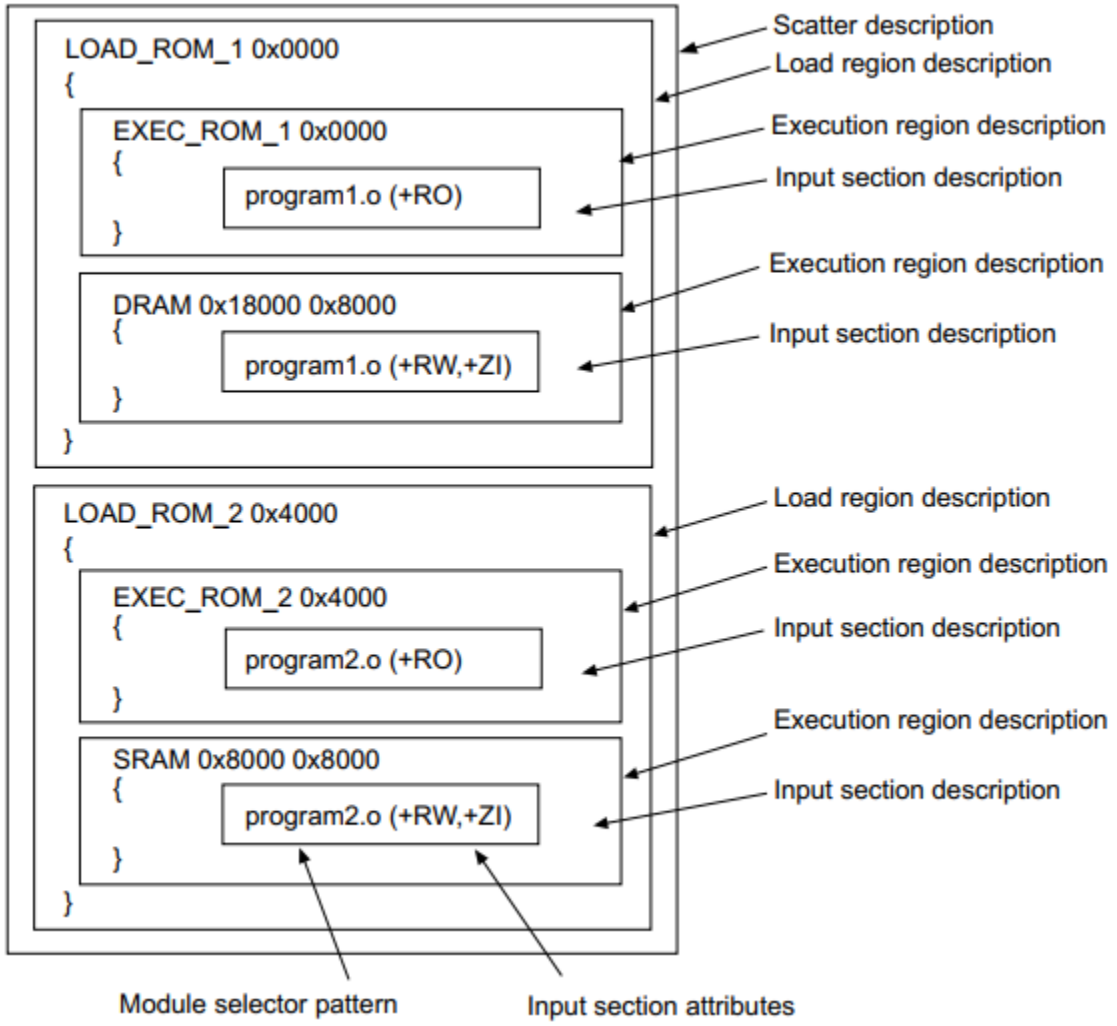
3. Scatter file – Load view and execution view

- **Comparing load and execution views**

Load	Description	Execution	Description
Load address	The address where a section or region is loaded into memory before the image containing it starts executing. The load address of a section or a non-root region can differ from its execution address.	Execution address	The address where a section or region is located while the image containing it is being executed.
Load region	A load region describes the layout of a contiguous chunk of memory in load address space.	Execution region	An execution region describes the layout of a contiguous chunk of memory in execution address space.

3. Scatter file – Syntax of a scatter file

- The following figure shows the components and organization of a typical scatter file:



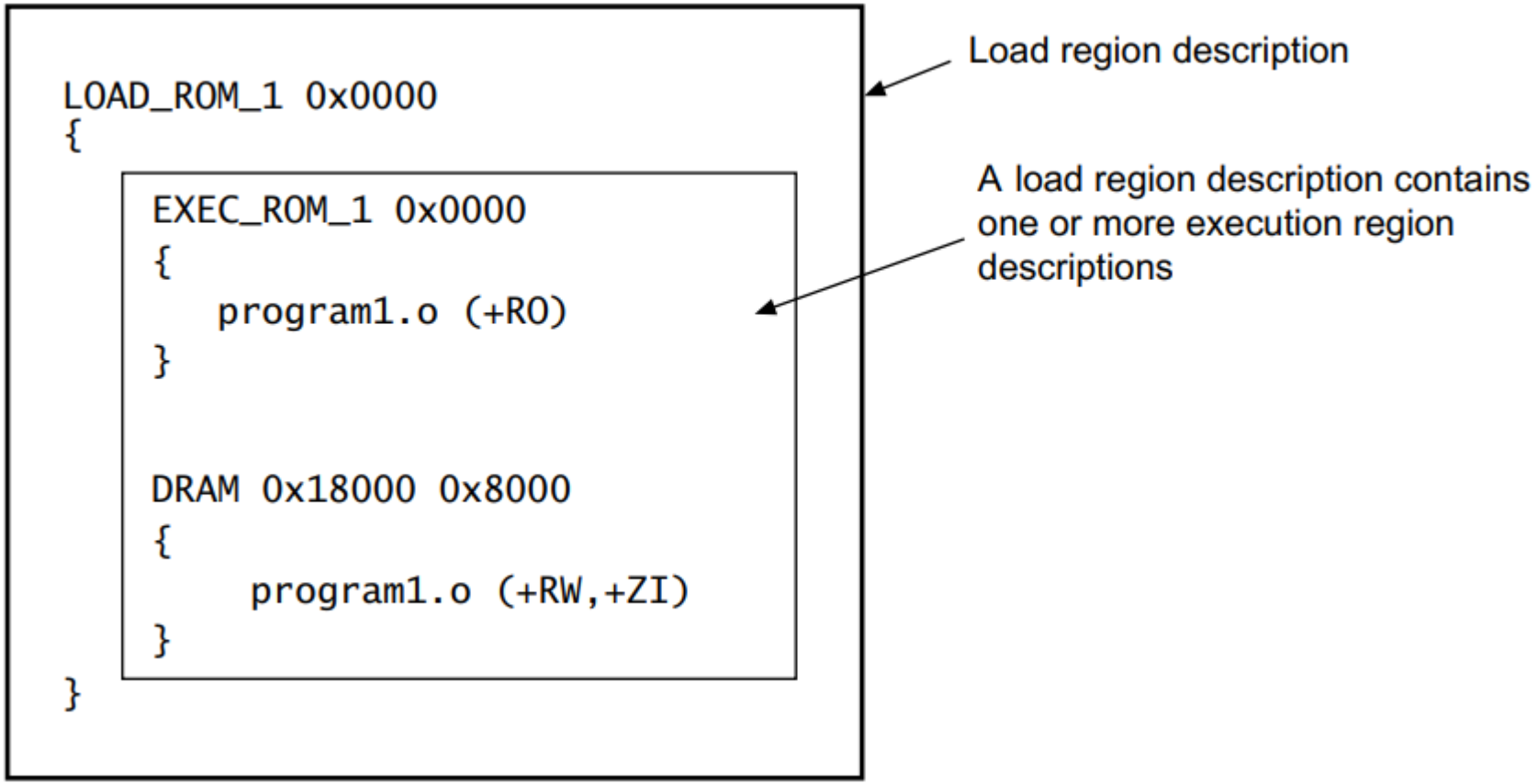
3. Scatter file – Syntax of a scatter file

- The BNF notation used in scatter-loading description syntax

Symbol	Description
"	Quotation marks indicate that a character that is normally part of the BNF syntax is used as a literal character in the definition. The definition B"+"C, for example, can only be replaced by the pattern B+C. The definition B+C can be replaced by, for example, patterns BC, BBC, or BBBC.
A ::= *B*	Defines *A* as *B*. For example, A::= B"+" | C means that *A* is equivalent to either B+ or C. The ::= notation defines a higher level construct in terms of its components. Each component might also have a ::= definition that defines it in terms of even simpler components. For example, A::= B and B::= C | D means that the definition *A* is equivalent to the patterns C or D.
[*A*]	Optional element *A*. For example, A::= B[C]D means that the definition *A* can be expanded into either BD or BCD.
A+	Element *A* can have one or more occurrences. For example, A::= B+ means that the definition *A* can be expanded into B, BB, or BBB.
*A**	Element *A* can have zero or more occurrences.
A | *B*	Either element *A* or *B* can occur, but not both.
(*A B*)	Element *A* and *B* are grouped together. This is particularly useful when the | operator is used or when a complex pattern is repeated. For example, A::=(B C)+ (D | E) means that the definition *A* can be expanded into any of BCD, BCE, BCBCD, BCBCE, BCBCBCD, or BCBCBCE.

3. Scatter file – Syntax of a scatter file

- **Load region descriptions:** A load region description specifies the region of memory where its child execution regions are to be placed.
- The following figure shows an example of a typical load region description:

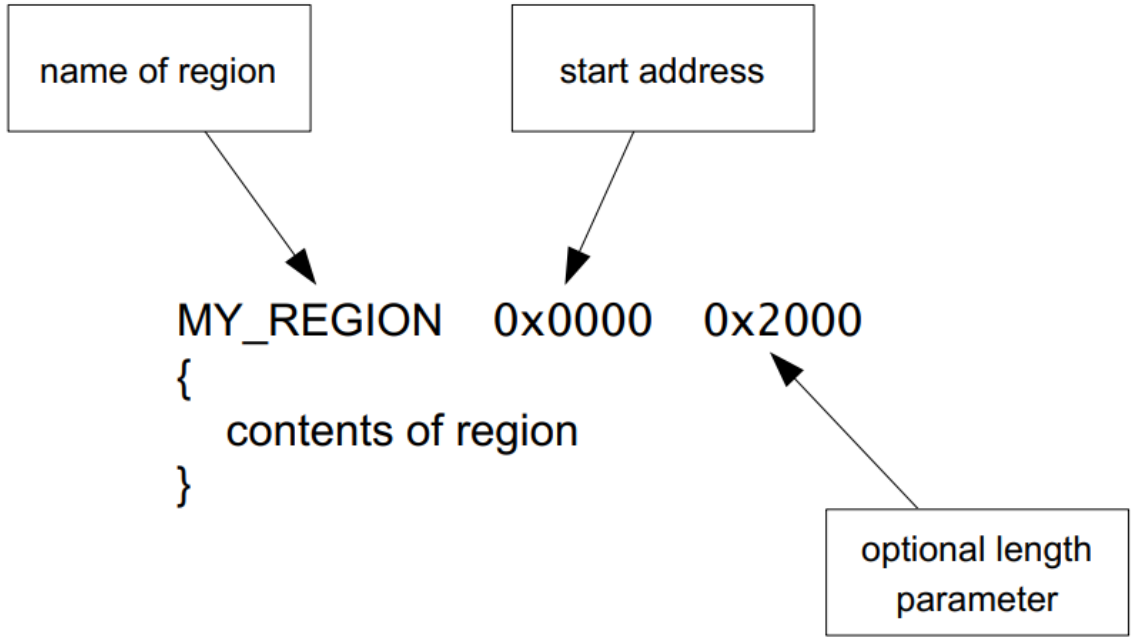


3. Scatter file – Syntax of a scatter file

- **Load region descriptions:** The syntax of a load region description, in *Backus-Naur Form* (BNF), is:

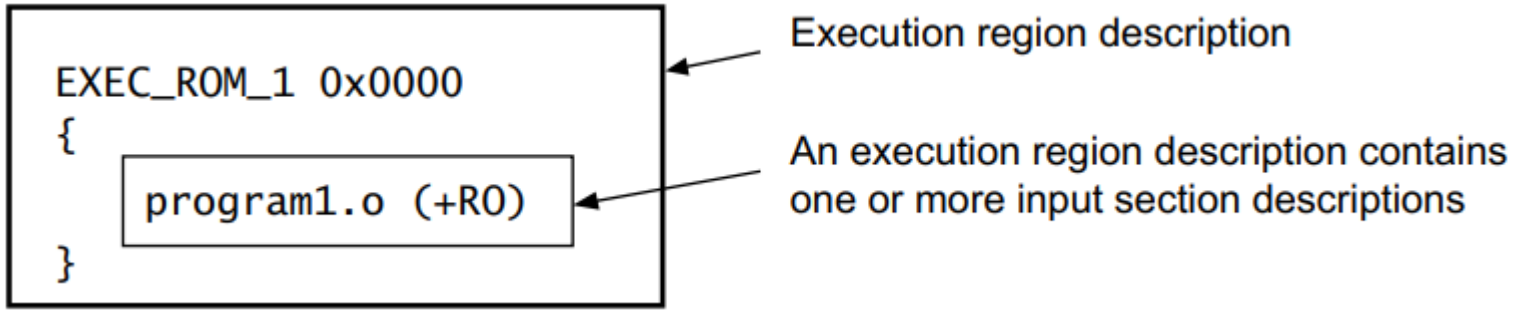
```
load_region_description ::=
  load_region_name  (base_address  | ("+" offset)) [attribute_list] [max_size]
       "{"
          execution_region_description+
       "}"
```

- For example:



3. Scatter file – Syntax of a scatter file

- **Execution region descriptions:**
- The following figure shows the components of a typical execution region description:



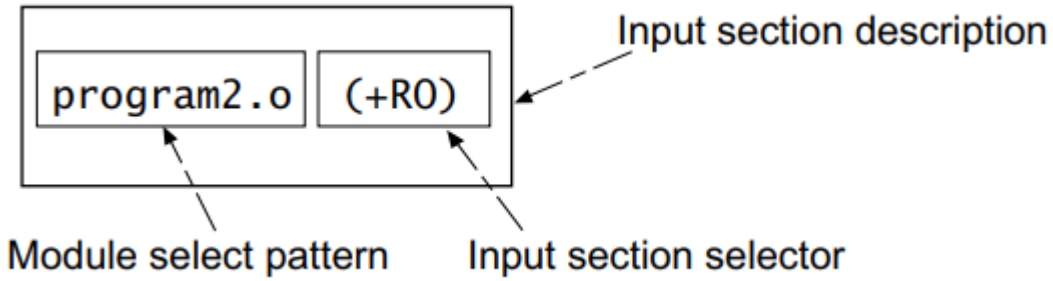
- The following figure shows the components and organization of a typical scatter file:

```
execution_region_description ::=
  exec_region_name (base_address | "+" offset) [attribute_list] [max_size | length]
        "{"
            input_section_description*
        "}"
```

3. Scatter file – Syntax of a scatter file

❖ Input section descriptions

- Components of an input section description:



- These many ways to set the section into the region. For example:
  - main.o(+ RO): match the section has the attribute RO into the region. The section attribute can be: RO-CODE, RO-DATA, RO(selects both RO-CODE and RO-DATA), RW-DATA, RW-CODE, RW(selects both RW-CODE and RW-DATA),XO, ZI, ENTRY(a section containing an ENTRY point).
  - Using the wildcard character * matches zero or more characters and ? matches any single character.For example:
    *.o: match all objects into the region.
    *.bss: mach all section .bss in to the region.
  - Using .ANY to place sections that are not automatically assigned to specific execution regions. For example: .ANY(+RW).

3. Scatter file – Syntax of a scatter file

- Sample scatter file:

```
#! armclang --target=arm-arm-none-eabi -mcpu=cortex-m3 -E -xc
; *************************************************************
; *** Scatter-Loading Description File generated by uVision ***
; *************************************************************
#define ROM_BASE 0x08000000
#define ROM_SIZE 0x00010000
#define RAM_BASE 0x20000000
#define RAM_SIZE 0x00005000
#define TOP_STACK ( RAM_BASE + RAM_SIZE )
#define STACK_SIZE 0x200

; Load region start form 0x08000000
LR_IROM1 ROM_BASE ROM_SIZE
{
  ; Execute region start form 0x08000000
  ER_IROM1 ROM_BASE
  {
    ;All library sections that must be in a
    ; root region, for example, __main.o,
    ; __scatter*.o, __dc*.o, and *Region$$Table
    *(InRoot$$Sections)

    ; The +RO attribute: RO-CODE and RO-DATA
    .ANY (+RO)

    ; Execute only section
    .ANY (+XO)
  }

  ; Ram region start from 0x20000000
  RW_IRAM1 RAM_BASE RAM_SIZE
  {
    ; Read write section and the zero-init section
    .ANY (+RW +ZI)
  }

  ARM_LIB_STACK TOP_STACK EMPTY -STACK_SIZE ; Stack region growing down
  { }
  ARM_LIB_HEAP +0 EMPTY 0 ; Heap region growing up
  { }
}

```

3. Scatter file

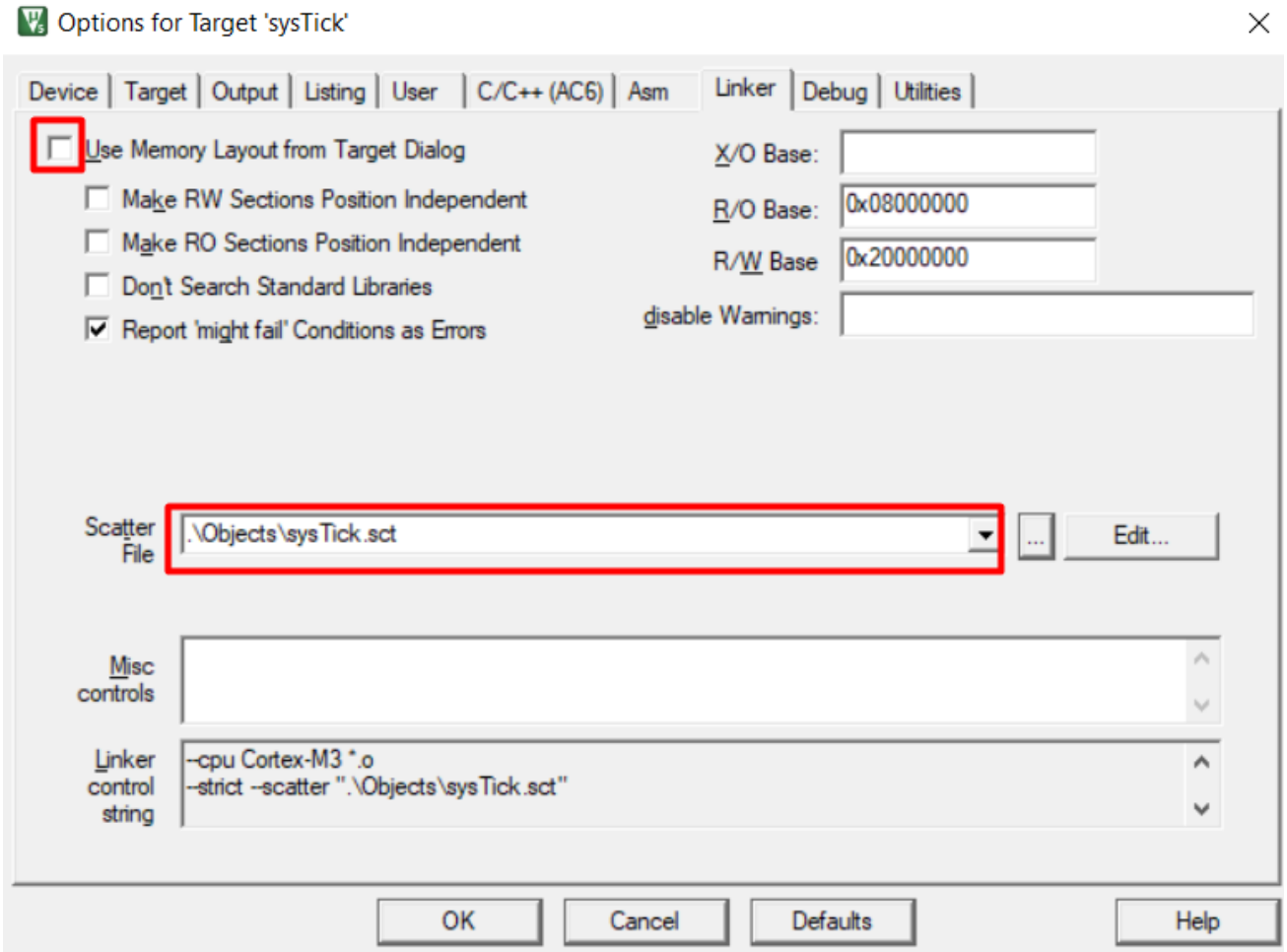
- Using the addition command-line option: --scatter=filename to build the image
- The command line:

./armlink --cpu=cortex-m4 --scatter=C:/cygwin/cygwin/home/Compiler/Linker.cst --entry=main --verbose --map --info=sizes,unused,unusedsymbols --symbols --load_addr_map_info --list=C:/cygwin/cygwin/home/Compiler/Infor.map C:/cygwin/cygwin/home/Compiler/main.o C:/cygwin/cygwin/home/Compiler/lib.o -o C:/cygwin/cygwin/home/Compiler/product.axf

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armlink --cpu=cortex-m4 --scatter=C:/cygwin/cygwin/home/Compiler/Linker.cst --entry=main --verbose --map --info=siz
es,unused,unusedsymbols --symbols --load_addr_map_info --list=C:/cygwin/cygwin/home/Compiler/Infor.map C:/cygwin/cygwin
/home/Compiler/main.o C:/cygwin/cygwin/home/Compiler/lib.o -o C:/cygwin/cygwin/home/Compiler/product.axf
```

3. Scatter file

- Modify scatter file in KeilC:



4. Memory map

- **Default memory map**
  - In an image where user have not described the memory map, the linker places code and data according to a default memory map.
  - The default memory map is described as follows:
    • The image is linked to load and run at address 0x8000. All *read-only* (RO) sections are placed first, followed by *read/write* (RW) sections, then *zero-initialized* (ZI) sections.
    • The heap follows directly on from the top of ZI, so the exact location is decided at link time.
    • The stack base location is provided by a semihosting operation during application startup. The value that this semihosting operation returns depends on the debug environment.

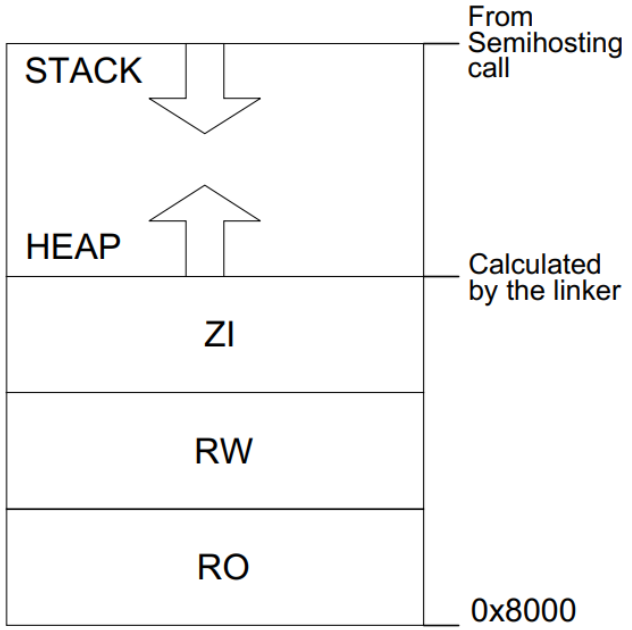
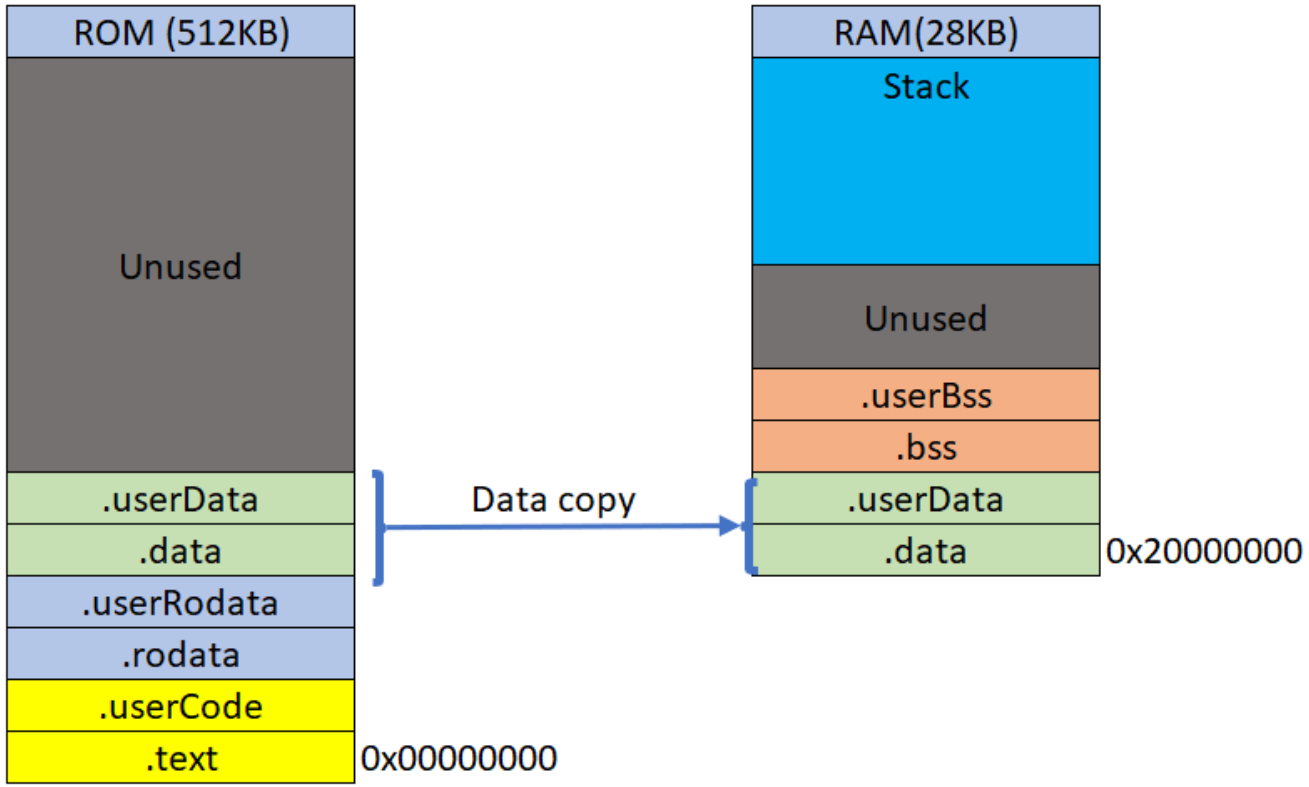


Figure 10-2 Default memory map

4. Memory map

- The user can modify the source code to create the user section by using the specific keyword (__attribute__, pragma, …), then with the scatter file, the user can create the user image.
- For example for the user image:



- Exercise: Create the image following the above image.


Thank you